



Set working directory

Make sure the notebook runs from the project root so relative paths like `data/raw/...` work correctly.

```
In [2]: import os
os.chdir(r"C:\Users\benetic handrach\indian_food_classifier")
print(os.getcwd())
```

C:\Users\benetic handrach\indian_food_classifier

Importing libraries

`os` and `shutil` for file handling, `random` for shuffling, `Path` for clean path management.

```
In [3]: import os
import shutil
import random
from pathlib import Path
```

Set random seed for reproducibility

Ensures the train/val/test split is the same every time we re-run this notebook.

```
In [4]: random.seed(42)
```

Define source and output paths

`SOURCE_DIR` points to the extracted dataset. `OUT_DIR` is where the split folders will be created.

```
In [5]: SOURCE_DIR = Path("data/raw/Indian Food Images/Indian Food Images")
OUT_DIR = Path("data/split")
```

Define split ratios

70% train, 15% validation, 15% test.

```
In [6]: TRAIN_RATIO = 0.70
VAL_RATIO = 0.15
TEST_RATIO = 0.15
```

List all class folders

Each subfolder inside `SOURCE_DIR` is one food category (class).

```
In [7]: classes = [d.name for d in SOURCE_DIR.iterdir() if d.is_dir()]
print(f"Found {len(classes)} classes")
print(classes[:5])
```

Found 74 classes

```
['adhirasam', 'aloo_gobi', 'aloo_matar', 'aloo_methi', 'aloo_shimla_mirc
h']
```

Create output folders

Make `train`, `val`, `test` folders inside `data/split`.

```
In [8]: for split in ["train", "val", "test"]:
        (OUT_DIR / split).mkdir(parents=True, exist_ok=True)

print("Created data/split/train, val, test")
```

Created data/split/train, val, test

Split and copy images

For each class, shuffle its images and copy them into train/val/test folders according to the ratios above.

```
In [9]: for cls in classes:
        cls_dir = SOURCE_DIR / cls
        images = list(cls_dir.glob("*.jpg"))
        random.shuffle(images)

        n = len(images)
        n_train = int(n * TRAIN_RATIO)
        n_val = int(n * VAL_RATIO)

        splits = {
            "train": images[:n_train],
            "val": images[n_train:n_train + n_val],
            "test": images[n_train + n_val:],
        }

        for split, files in splits.items():
            split_cls_dir = OUT_DIR / split / cls
            split_cls_dir.mkdir(parents=True, exist_ok=True)
            for f in files:
                shutil.copy(f, split_cls_dir / f.name)

print("Split complete.")
```

Split complete.

Verify the split

Count classes and images in each split folder to confirm everything copied correctly.

```
In [10... for split in ["train", "val", "test"]:  
    p = OUT_DIR / split  
    n_classes = len(os.listdir(p))  
    n_images = sum(len(os.listdir(p / c)) for c in os.listdir(p))  
    print(f"{split} -> classes: {n_classes}, images: {n_images}")
```

train -> classes: 74, images: 2558

val -> classes: 74, images: 511

test -> classes: 74, images: 586

Importing libraries

- `torch` — core tensor library
- `torchvision.datasets.ImageFolder` — a ready-made Dataset class that assumes your folder structure is `root/<class_name>/<image>.jpg` (which is exactly what we built in Step 2)
- `torchvision.transforms` — defines how each image gets resized/normalized/augmented before it reaches the model
- `DataLoader` — batches and shuffles the dataset during training

```
In [2]: import sys  
print(sys.executable)
```

c:\Users\benmetic handrach\miniconda3\envs\tina\python.exe

```
In [2]: %pip show torch
```

Note: you may need to restart the kernel to use updated packages.

```
In [3]: import torch
print("Torch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
```

Torch version: 2.11.0+cu128

CUDA available: True

GPU name: NVIDIA GeForce RTX 3050 6GB Laptop GPU

Re-importing core libraries

The kernel was restarted after fixing the PyTorch/CUDA install, so all previous imports and variables are gone from memory. We re-import everything needed for the dataset pipeline.

```
In [4]: import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

Setting the image size

We resize every image to 224x224 pixels. This is the standard input size expected by pretrained ImageNet models like EfficientNet, which we'll use later for transfer learning — matching this size means the pretrained weights work correctly on our data.

```
In [5]: IMG_SIZE = 224
```

Defining normalization values

These are the exact mean and standard deviation (per RGB channel) that ImageNet-pretrained models were trained with. Using these same values ensures our images are scaled the way the pretrained EfficientNet expects, rather than computing our own statistics from scratch.

```
In [6]: IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]
```

Defining transforms for training vs. evaluation

The training transform includes random flips, rotation, and color jitter (data augmentation) so the model sees varied versions of each image and generalizes better instead of memorizing exact pixels. The evaluation

transform (used for validation and test) skips augmentation, since we want to measure performance on clean, unmodified images.

```
In [7]: train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

eval_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])
```

Set working directory

The kernel was restarted, which resets the working directory. We need to point it back to the project root so relative paths like `data/split/train` resolve correctly.

```
In [11... import os
os.chdir(r"C:\Users\benetic handrach\indian_food_classifier")
print(os.getcwd())
```

C:\Users\benetic handrach\indian_food_classifier

Diagnose: check image counts for the problem class

Before patching anything, let's confirm the class `sheer_korma` genuinely has too few images, and check if other classes have the same issue.

```
In [13... from pathlib import Path

SOURCE_DIR = Path("data/raw/Indian Food Images/Indian Food Images")

counts = {}
for cls_dir in SOURCE_DIR.iterdir():
    if cls_dir.is_dir():
        n = len(list(cls_dir.glob("*.jpg")))
        counts[cls_dir.name] = n

# Sort by count, show the smallest classes
sorted_counts = sorted(counts.items(), key=lambda x: x[1])
for cls, n in sorted_counts[:10]:
    print(cls, "->", n, "images")
```

```
sheer_korma -> 5 images
adhirasam -> 50 images
aloo_gobi -> 50 images
aloo_matar -> 50 images
aloo_methi -> 50 images
aloo_shimla_mirch -> 50 images
aloo_tikki -> 50 images
anarsa -> 50 images
ariselu -> 50 images
bandar_laddu -> 50 images
```

Diagnose: find ALL classes with too few images

Rather than fixing this one class in isolation, let's check the full distribution to see how many classes are affected, so we fix the split logic once for all of them.

```
In [15... low_count_classes = {cls: n for cls, n in counts.items() if n < 10}
print(f"Classes with fewer than 10 images: {len(low_count_classes)}")
for cls, n in low_count_classes.items():
    print(cls, "->", n)
```

```
Classes with fewer than 10 images: 1
sheer_korma -> 5
```

Why this matters, and the fix

A class with only 5 images can't meaningfully support a 70/15/15 split — even a perfect model can't "learn" a food from 3 training images and be validated on 0-1. For a real classification project, the standard practice is to **drop classes that don't have enough images** (typically we need at least ~10-15 to get any reasonable train/val/test signal) rather than force-fitting them into a split, which would just introduce noisy near-empty folders.

We'll rewrite Step 2's split logic with two fixes:

1. Skip any class with fewer than a minimum threshold (we'll use 10) — report which classes get dropped
2. For the remaining classes, guarantee **at least 1 image in each of train/val/test** using explicit allocation instead of relying on `int(n * ratio)` rounding, which can silently produce zero

```
In [16... import shutil
import random
from pathlib import Path

random.seed(42)
```

```

SOURCE_DIR = Path("data/raw/Indian Food Images/Indian Food Images")
OUT_DIR = Path("data/split")

MIN_IMAGES_PER_CLASS = 10
TRAIN_RATIO = 0.70
VAL_RATIO = 0.15
# TEST_RATIO is whatever remains

# Clear out the old (broken) split so we start fresh
if OUT_DIR.exists():
    shutil.rmtree(OUT_DIR)

for split in ["train", "val", "test"]:
    (OUT_DIR / split).mkdir(parents=True, exist_ok=True)

dropped_classes = []
kept_classes = []

for cls_dir in sorted(SOURCE_DIR.iterdir()):
    if not cls_dir.is_dir():
        continue

    images = list(cls_dir.glob("*.jpg"))
    n = len(images)

    if n < MIN_IMAGES_PER_CLASS:
        dropped_classes.append((cls_dir.name, n))
        continue

    kept_classes.append(cls_dir.name)
    random.shuffle(images)

    n_val = max(1, int(n * VAL_RATIO))
    n_train = max(1, int(n * TRAIN_RATIO))
    # ensure we don't allocate more than we have
    n_train = min(n_train, n - n_val - 1) # leave at least 1 for test
    n_test = n - n_train - n_val

    splits = {
        "train": images[:n_train],
        "val": images[n_train:n_train + n_val],
        "test": images[n_train + n_val:],
    }

    for split, files in splits.items():
        split_cls_dir = OUT_DIR / split / cls_dir.name
        split_cls_dir.mkdir(parents=True, exist_ok=True)
        for f in files:
            shutil.copy(f, split_cls_dir / f.name)

print(f"Kept {len(kept_classes)} classes")
print(f"Dropped {len(dropped_classes)} classes (fewer than {MIN_IMAGES_PER_CLASS})")
for cls, n in dropped_classes:

```

```
print(f" {cls} ({n} images)")
```

Kept 73 classes

Dropped 1 classes (fewer than 10 images):

sheer_korma (5 images)

Verify the corrected split

Every class should now have at least 1 image in train, val, and test — no empty folders.

In [17... `import os`

```
for split in ["train", "val", "test"]:
    p = OUT_DIR / split
    n_classes = len(os.listdir(p))
    n_images = sum(len(os.listdir(p / c)) for c in os.listdir(p))
    print(f"{split} -> classes: {n_classes}, images: {n_images}")
```

train -> classes: 73, images: 2555

val -> classes: 73, images: 511

test -> classes: 73, images: 584

Loading the datasets

`ImageFolder` automatically reads our `data/split/train`, `val`, and `test` folders (created in Step 2), treating each subfolder name as a class label.

Augmentation is applied only to the training set.

In [18... `train_dataset = datasets.ImageFolder("data/split/train", transform=train_`
`val_dataset = datasets.ImageFolder("data/split/val", transform=eval_tra`
`test_dataset = datasets.ImageFolder("data/split/test", transform=eval_tr`

```
print("Train size:", len(train_dataset))
print("Val size:", len(val_dataset))
print("Test size:", len(test_dataset))
print("Number of classes:", len(train_dataset.classes))
```

Train size: 2555

Val size: 511

Test size: 584

Number of classes: 73

In [19... `BATCH_SIZE = 32`

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

In [20... `images, labels = next(iter(train_loader))`
`print("Image batch shape:", images.shape)`

```
print("Label batch shape:", labels.shape)
```

```
Image batch shape: torch.Size([32, 3, 224, 224])  
Label batch shape: torch.Size([32])
```

```
In [21... device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print("Using device:", device)
```

```
Using device: cuda
```

Importing model-building tools

`torch.nn` gives us layers (Linear, ReLU, etc.) and loss functions.

`torch.optim` gives us optimizers (SGD, Adam, RMSprop) — directly tied to your syllabus's "weight optimization techniques" topic.

```
In [22... import torch.nn as nn  
import torch.optim as optim
```

Defining the ANN architecture

This is a plain fully-connected network:

- **Flatten:** converts each `[3, 224, 224]` image into a single vector of `3*224*224 = 150,528` numbers
- **Linear layers:** each neuron computes a weighted sum of all inputs plus a bias — this is the "artificial neuron" from your syllabus, directly contrasted with biological neurons (which fire based on threshold potentials, not simple weighted sums)
- **ReLU activation:** introduces non-linearity between layers. Without this, stacking Linear layers would mathematically collapse into a single Linear layer, no matter how many we stack — activation functions are what let the network learn non-linear patterns
- **Output layer:** has as many neurons as classes (73), each producing a raw score ("logit") for that class

```
In [23... class SimpleANN(nn.Module):  
    def __init__(self, num_classes, img_size=224):  
        super().__init__()  
        input_dim = 3 * img_size * img_size  
  
        self.model = nn.Sequential(  
            nn.Flatten(),  
            nn.Linear(input_dim, 512),  
            nn.ReLU(),  
            nn.Linear(512, 256),  
            nn.ReLU(),
```

```

        nn.Linear(256, num_classes)
    )

    def forward(self, x):
        return self.model(x)

```

Instantiating the model and moving it to the GPU

`num_classes` comes from our dataset (73). We call `.to(device)` to move all model weights onto the GPU, so training uses your RTX 3050 instead of the CPU.

```

In [24... num_classes = len(train_dataset.classes)

ann_model = SimpleANN(num_classes=num_classes).to(device)
print(ann_model)

SimpleANN(
  (model): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=150528, out_features=512, bias=True)
    (2): ReLU()
    (3): Linear(in_features=512, out_features=256, bias=True)
    (4): ReLU()
    (5): Linear(in_features=256, out_features=73, bias=True)
  )
)

```

Defining the loss function and optimizer

- **CrossEntropyLoss**: the standard loss for multi-class classification. It combines a softmax (turns raw logits into probabilities) with negative log-likelihood (penalizes confident wrong predictions heavily).
- **Optimizer — Adam**: this is one of the "weight optimization techniques" from your syllabus. Adam adapts the learning rate per parameter using running estimates of gradient mean and variance, which generally converges faster and more reliably than plain SGD. We'll compare optimizers explicitly later (SGD vs Adam vs RMSprop) as one of your lab deliverables.

```

In [25... criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(ann_model.parameters(), lr=0.001)

```

Writing the training loop

For each epoch, we loop through every batch in `train_loader`:

1. Move the batch to GPU
2. Forward pass: get model predictions
3. Compute loss (how wrong the predictions are)
4. Backward pass: compute gradients via backpropagation (Unit 2 concept, but the mechanism is used here too)
5. Optimizer step: update weights using those gradients
6. Zero the gradients before the next batch (PyTorch accumulates gradients by default, so we must reset them)

We also track running loss and accuracy to monitor progress.

```
In [26... def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_acc = correct / total
    return epoch_loss, epoch_acc
```

Writing the validation loop

Similar to training, but we don't compute gradients (`torch.no_grad()`) or update weights — we're only measuring how well the model generalizes to unseen data.

```
In [27... def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
```

```

for images, labels in loader:
    images, labels = images.to(device), labels.to(device)

    outputs = model(images)
    loss = criterion(outputs, labels)

    running_loss += loss.item() * images.size(0)
    _, predicted = outputs.max(1)
    correct += predicted.eq(labels).sum().item()
    total += labels.size(0)

epoch_loss = running_loss / total
epoch_acc = correct / total
return epoch_loss, epoch_acc

```

Running the training loop for several epochs

We train for 15 epochs, printing train/val loss and accuracy after each. We also store the history so we can plot it afterward — useful for spotting overfitting (train accuracy climbing while val accuracy stalls or drops).

```

In [29... EPOCHS = 15
history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc":

for epoch in range(EPOCHS):
    train_loss, train_acc = train_one_epoch(ann_model, train_loader, crit
    val_loss, val_acc = evaluate(ann_model, val_loader, criterion, device

    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)

    print(f"Epoch {epoch+1}/{EPOCHS} | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

```

```

Epoch 1/15 | Train Loss: 4.2906, Train Acc: 0.0188 | Val Loss: 4.3164, Val Acc: 0.0176
Epoch 2/15 | Train Loss: 4.2851, Train Acc: 0.0204 | Val Loss: 4.4097, Val Acc: 0.0137
Epoch 3/15 | Train Loss: 4.2937, Train Acc: 0.0192 | Val Loss: 4.3381, Val Acc: 0.0137
Epoch 4/15 | Train Loss: 4.2774, Train Acc: 0.0200 | Val Loss: 4.3283, Val Acc: 0.0157
Epoch 5/15 | Train Loss: 4.3109, Train Acc: 0.0180 | Val Loss: 4.2916, Val Acc: 0.0137
Epoch 6/15 | Train Loss: 4.2901, Train Acc: 0.0157 | Val Loss: 4.2855, Val Acc: 0.0137
Epoch 7/15 | Train Loss: 4.3034, Train Acc: 0.0164 | Val Loss: 4.2982, Val Acc: 0.0137
Epoch 8/15 | Train Loss: 4.2784, Train Acc: 0.0184 | Val Loss: 4.4043, Val Acc: 0.0157
Epoch 9/15 | Train Loss: 4.2894, Train Acc: 0.0176 | Val Loss: 4.3119, Val Acc: 0.0157
Epoch 10/15 | Train Loss: 4.2852, Train Acc: 0.0207 | Val Loss: 4.3475, Val Acc: 0.0137
Epoch 11/15 | Train Loss: 4.2875, Train Acc: 0.0168 | Val Loss: 4.2904, Val Acc: 0.0137
Epoch 12/15 | Train Loss: 4.2844, Train Acc: 0.0153 | Val Loss: 4.3208, Val Acc: 0.0137
Epoch 13/15 | Train Loss: 4.2732, Train Acc: 0.0180 | Val Loss: 4.3386, Val Acc: 0.0137
Epoch 14/15 | Train Loss: 4.2816, Train Acc: 0.0160 | Val Loss: 4.2906, Val Acc: 0.0176
Epoch 15/15 | Train Loss: 4.2813, Train Acc: 0.0133 | Val Loss: 4.2837, Val Acc: 0.0157

```

Defining the MLP architecture

Compared to the ANN, we insert `BatchNorm1d` after each Linear layer (before the activation) and `Dropout` after the activation. `BatchNorm1d` is used here (not `BatchNorm2d`) because our data at this point is a flat 1D vector per sample, not a 2D feature map — `BatchNorm2d` will come later with the CNN.

```

In [30... class MLP(nn.Module):
    def __init__(self, num_classes, img_size=224, dropout_p=0.3):
        super().__init__()
        input_dim = 3 * img_size * img_size

        self.model = nn.Sequential(
            nn.Flatten(),

            nn.Linear(input_dim, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(dropout_p),

            nn.Linear(512, 256),

```

```

        nn.BatchNorm1d(256),
        nn.ReLU(),
        nn.Dropout(dropout_p),

        nn.Linear(256, num_classes)
    )

    def forward(self, x):
        return self.model(x)

```

Instantiating the MLP model

Same pattern as the ANN — create the model, move it to GPU.

```

In [31... mlp_model = MLP(num_classes=num_classes).to(device)
print(mlp_model)

MLP(
  (model): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=150528, out_features=512, bias=True)
    (2): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_run
ning_stats=True)
    (3): ReLU()
    (4): Dropout(p=0.3, inplace=False)
    (5): Linear(in_features=512, out_features=256, bias=True)
    (6): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_run
ning_stats=True)
    (7): ReLU()
    (8): Dropout(p=0.3, inplace=False)
    (9): Linear(in_features=256, out_features=73, bias=True)
  )
)

```

Loss function and optimizer for the MLP

Same CrossEntropyLoss as before. We reuse Adam here too, so the ANN-vs-MLP comparison isolates the effect of architecture (BatchNorm/Dropout) rather than mixing in a different optimizer's effect.

```

In [32... mlp_criterion = nn.CrossEntropyLoss()
mlp_optimizer = optim.Adam(mlp_model.parameters(), lr=0.001)

```

Training the MLP

We reuse the same `train_one_epoch` and `evaluate` functions from Step 4 — this is why we wrote them as reusable functions rather than one-off code. Same 15 epochs, same data, only the model differs.

```
In [34... EPOCHS = 15
mlp_history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_ac

for epoch in range(EPOCHS):
    train_loss, train_acc = train_one_epoch(mlp_model, train_loader, mlp_
    val_loss, val_acc = evaluate(mlp_model, val_loader, mlp_criterion, de

    mlp_history["train_loss"].append(train_loss)
    mlp_history["train_acc"].append(train_acc)
    mlp_history["val_loss"].append(val_loss)
    mlp_history["val_acc"].append(val_acc)

    print(f"Epoch {epoch+1}/{EPOCHS} | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")
```

```
Epoch 1/15 | Train Loss: 4.3018, Train Acc: 0.0250 | Val Loss: 4.1383, Va
l Acc: 0.0450
Epoch 2/15 | Train Loss: 4.0868, Train Acc: 0.0431 | Val Loss: 4.0419, Va
l Acc: 0.0548
Epoch 3/15 | Train Loss: 3.9916, Train Acc: 0.0568 | Val Loss: 3.9773, Va
l Acc: 0.0705
Epoch 4/15 | Train Loss: 3.8896, Train Acc: 0.0751 | Val Loss: 3.9083, Va
l Acc: 0.0802
Epoch 5/15 | Train Loss: 3.7971, Train Acc: 0.0900 | Val Loss: 3.8501, Va
l Acc: 0.0802
Epoch 6/15 | Train Loss: 3.7123, Train Acc: 0.0998 | Val Loss: 3.7959, Va
l Acc: 0.0900
Epoch 7/15 | Train Loss: 3.6216, Train Acc: 0.1155 | Val Loss: 3.7743, Va
l Acc: 0.1155
Epoch 8/15 | Train Loss: 3.5760, Train Acc: 0.1186 | Val Loss: 3.7457, Va
l Acc: 0.1096
Epoch 9/15 | Train Loss: 3.4710, Train Acc: 0.1350 | Val Loss: 3.6802, Va
l Acc: 0.1057
Epoch 10/15 | Train Loss: 3.4396, Train Acc: 0.1401 | Val Loss: 3.7165, V
al Acc: 0.1115
Epoch 11/15 | Train Loss: 3.3806, Train Acc: 0.1566 | Val Loss: 3.7129, V
al Acc: 0.1213
Epoch 12/15 | Train Loss: 3.3220, Train Acc: 0.1652 | Val Loss: 3.6856, V
al Acc: 0.1331
Epoch 13/15 | Train Loss: 3.2560, Train Acc: 0.1914 | Val Loss: 3.7351, V
al Acc: 0.1272
Epoch 14/15 | Train Loss: 3.1940, Train Acc: 0.1977 | Val Loss: 3.7357, V
al Acc: 0.1037
Epoch 15/15 | Train Loss: 3.1487, Train Acc: 0.2051 | Val Loss: 3.7161, V
al Acc: 0.1174
```

Defining the CNN architecture

We stack three convolutional blocks (Conv → BatchNorm → ReLU → MaxPool), each block doubling the number of filters (32 → 64 → 128) while pooling halves the spatial dimensions. After the conv blocks, we flatten and pass

through fully connected layers to produce class scores — this is the "fully connected layers" role your syllabus mentions: combining the extracted features into a final classification decision.

With input 224x224, three rounds of 2x2 pooling give us $224 \rightarrow 112 \rightarrow 56 \rightarrow 28$, so the flattened feature map size is `128 * 28 * 28`.

```
In [35... class SimpleCNN(nn.Module):
    def __init__(self, num_classes, dropout_p=0.4):
        super().__init__()

        self.conv_block = nn.Sequential(
            # Block 1
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2), # 224 -> 112

            # Block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2), # 112 -> 56

            # Block 3
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2), # 56 -> 28
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 28 * 28, 256),
            nn.ReLU(),
            nn.Dropout(dropout_p),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.conv_block(x)
        x = self.classifier(x)
        return x
```

Instantiating the CNN

Same pattern as before — create, move to GPU.

```
In [36... cnn_model = SimpleCNN(num_classes=num_classes).to(device)
print(cnn_model)
```

```

SimpleCNN(
    (conv_block): Sequential(
      (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU()
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (6): ReLU()
      (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (10): ReLU()
      (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (classifier): Sequential(
      (0): Flatten(start_dim=1, end_dim=-1)
      (1): Linear(in_features=100352, out_features=256, bias=True)
      (2): ReLU()
      (3): Dropout(p=0.4, inplace=False)
      (4): Linear(in_features=256, out_features=73, bias=True)
    )
  )

```

Loss function and optimizer

This time we'll actually use the training loop to compare optimizers, as your syllabus lab exercise asks (SGD vs Adam vs RMSprop). We start with Adam here, matching the earlier two models, so the CNN-vs-MLP-vs-ANN comparison is fair (architecture is the only thing changing).

```

In [37... cnn_criterion = nn.CrossEntropyLoss()
cnn_optimizer = optim.Adam(cnn_model.parameters(), lr=0.001)

```

Training the CNN

Same reusable `train_one_epoch / evaluate` functions. CNNs are heavier to compute than the flat ANN/MLP, so expect each epoch to take noticeably longer — but accuracy should be substantially better.

```

In [38... EPOCHS = 15

```

```

cnn_history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}

for epoch in range(EPOCHS):
    train_loss, train_acc = train_one_epoch(cnn_model, train_loader, cnn_criterion, device)
    val_loss, val_acc = evaluate(cnn_model, val_loader, cnn_criterion, device)

    cnn_history["train_loss"].append(train_loss)
    cnn_history["train_acc"].append(train_acc)
    cnn_history["val_loss"].append(val_loss)
    cnn_history["val_acc"].append(val_acc)

    print(f"Epoch {epoch+1}/{EPOCHS} | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

```

```

Epoch 1/15 | Train Loss: 7.2229, Train Acc: 0.0145 | Val Loss: 4.2910, Val Acc: 0.0137
Epoch 2/15 | Train Loss: 4.2915, Train Acc: 0.0137 | Val Loss: 4.2909, Val Acc: 0.0137
Epoch 3/15 | Train Loss: 4.2915, Train Acc: 0.0121 | Val Loss: 4.2909, Val Acc: 0.0137
Epoch 4/15 | Train Loss: 4.2914, Train Acc: 0.0137 | Val Loss: 4.2909, Val Acc: 0.0137
Epoch 5/15 | Train Loss: 4.2914, Train Acc: 0.0090 | Val Loss: 4.2908, Val Acc: 0.0137
Epoch 6/15 | Train Loss: 4.2913, Train Acc: 0.0137 | Val Loss: 4.2908, Val Acc: 0.0137
Epoch 7/15 | Train Loss: 4.2913, Train Acc: 0.0137 | Val Loss: 4.2908, Val Acc: 0.0137
Epoch 8/15 | Train Loss: 4.2913, Train Acc: 0.0102 | Val Loss: 4.2907, Val Acc: 0.0137
Epoch 9/15 | Train Loss: 4.2913, Train Acc: 0.0114 | Val Loss: 4.2907, Val Acc: 0.0137
Epoch 10/15 | Train Loss: 4.3010, Train Acc: 0.0090 | Val Loss: 4.2907, Val Acc: 0.0137
Epoch 11/15 | Train Loss: 4.2913, Train Acc: 0.0137 | Val Loss: 4.2906, Val Acc: 0.0137
Epoch 12/15 | Train Loss: 4.2912, Train Acc: 0.0114 | Val Loss: 4.2906, Val Acc: 0.0137
Epoch 13/15 | Train Loss: 4.2912, Train Acc: 0.0114 | Val Loss: 4.2906, Val Acc: 0.0137
Epoch 14/15 | Train Loss: 4.2912, Train Acc: 0.0121 | Val Loss: 4.2906, Val Acc: 0.0137
Epoch 15/15 | Train Loss: 4.2912, Train Acc: 0.0078 | Val Loss: 4.2906, Val Acc: 0.0137

```

Importing the pretrained EfficientNet

`torchvision.models` ships EfficientNetB0 with pretrained ImageNet weights built in. We import the model and its associated weights enum.

In [39... `from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights`

Loading the pretrained model

`weights=EfficientNet_B0_Weights.DEFAULT` downloads the ImageNet-pretrained weights (first time only — cached afterward). Note: the pretrained model expects the exact same 224x224 size and ImageNet normalization we already set up in Step 3 — this is precisely why we used those values from the start.

```
In [40... weights = EfficientNet_B0_Weights.DEFAULT
effnet_model = efficientnet_b0(weights=weights)

Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwigh
an-7f5810bc.pth" to C:\Users\benetic handrach/.cache\torch\hub\checkpoi
ts\efficientnet_b0_rwightman-7f5810bc.pth
100.0%
```

Freezing the pretrained backbone

We set `requires_grad = False` on every existing parameter, which tells PyTorch not to compute gradients for them — they stay fixed at their pretrained ImageNet values. Only the new classification head we add next will actually learn during this first training stage.

```
In [41... for param in effnet_model.parameters():
    param.requires_grad = False
```

Replacing the classification head

EfficientNetB0's original final layer outputs 1000 scores (for ImageNet's 1000 classes). We replace it with a new `Linear` layer outputting 73 scores (our number of food classes). This new layer is created fresh, so by default `requires_grad = True` — it's the only part that will train in this first stage.

We also add Dropout before it, matching EfficientNet's own design convention, to reduce overfitting on our smaller dataset.

```
In [42... in_features = effnet_model.classifier[1].in_features

effnet_model.classifier = nn.Sequential(
    nn.Dropout(p=0.3, inplace=True),
    nn.Linear(in_features, num_classes)
)

effnet_model = effnet_model.to(device)
```

Loss function and optimizer

Note: we pass only `effnet_model.classifier.parameters()` to the optimizer, not the whole model — since the backbone is frozen, there's no point giving the optimizer parameters it can't update. This also makes training much faster.

Learning rate is slightly higher here (0.001) since we're training a small, freshly-initialized head from scratch.

```
In [43... effnet_criterion = nn.CrossEntropyLoss()
effnet_optimizer = optim.Adam(effnet_model.classifier.parameters(), lr=0.
```

Stage 1 training: feature extraction

Same reusable training loop. Expect this to be dramatically better than the from-scratch CNN, likely reaching much higher validation accuracy in far fewer epochs — that's the whole point of transfer learning. We'll do fewer epochs here (8) since the head is small and converges quickly.

```
In [44... EPOCHS_STAGE1 = 8
effnet_history = {"train_loss": [], "train_acc": [], "val_loss": [], "val

for epoch in range(EPOCHS_STAGE1):
    train_loss, train_acc = train_one_epoch(effnet_model, train_loader, e
    val_loss, val_acc = evaluate(effnet_model, val_loader, effnet_criteri

    effnet_history["train_loss"].append(train_loss)
    effnet_history["train_acc"].append(train_acc)
    effnet_history["val_loss"].append(val_loss)
    effnet_history["val_acc"].append(val_acc)

    print(f"[Stage 1] Epoch {epoch+1}/{EPOCHS_STAGE1} | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")
```

```
[Stage 1] Epoch 1/8 | Train Loss: 3.8722, Train Acc: 0.1542 | Val Loss: 3.3530, Val Acc: 0.3151
[Stage 1] Epoch 2/8 | Train Loss: 2.8744, Train Acc: 0.4427 | Val Loss: 2.8218, Val Acc: 0.4149
[Stage 1] Epoch 3/8 | Train Loss: 2.3308, Train Acc: 0.5342 | Val Loss: 2.5173, Val Acc: 0.4247
[Stage 1] Epoch 4/8 | Train Loss: 2.0192, Train Acc: 0.5836 | Val Loss: 2.3384, Val Acc: 0.4579
[Stage 1] Epoch 5/8 | Train Loss: 1.7870, Train Acc: 0.6243 | Val Loss: 2.2271, Val Acc: 0.4658
[Stage 1] Epoch 6/8 | Train Loss: 1.6323, Train Acc: 0.6560 | Val Loss: 2.1656, Val Acc: 0.4716
[Stage 1] Epoch 7/8 | Train Loss: 1.4646, Train Acc: 0.6900 | Val Loss: 2.0792, Val Acc: 0.4873
[Stage 1] Epoch 8/8 | Train Loss: 1.3765, Train Acc: 0.7010 | Val Loss: 2.0501, Val Acc: 0.5049
```

Unfreezing the last few blocks of EfficientNet

EfficientNetB0's `features` module is a sequence of blocks (roughly 9 stages). We unfreeze only the last 2-3 blocks, keeping earlier, more generic feature extractors frozen. This is a standard, conservative fine-tuning strategy.

```
In [45... # Check how many blocks exist
print(len(effnet_model.features))
```

9

Unfreeze the last 3 blocks

We set `requires_grad = True` only on the final 3 blocks of the feature extractor, plus the classifier head (already trainable from Stage 1).

```
In [46... for param in effnet_model.features[-3:].parameters():
    param.requires_grad = True

# Confirm how many parameters are now trainable
trainable = sum(p.numel() for p in effnet_model.parameters() if p.requires_grad)
total = sum(p.numel() for p in effnet_model.parameters())
print(f"Trainable parameters: {trainable:,} / {total:,}")
```

Trainable parameters: 3,249,253 / 4,101,061

New optimizer with a much smaller learning rate

This is the most important detail of fine-tuning: the learning rate must be small (here, 10x smaller than Stage 1). The backbone layers already hold well-trained ImageNet weights — a large learning rate would aggressively overwrite that knowledge instead of gently adapting it. We create a fresh

optimizer here since the set of trainable parameters has changed.

```
In [47... finetune_optimizer = optim.Adam(  
    filter(lambda p: p.requires_grad, effnet_model.parameters()),  
    lr=0.0001  
)
```

Stage 2 training: fine-tuning

Fewer epochs than Stage 1 (5) — fine-tuning needs less time since we're making small adjustments to already-good weights, not learning from scratch. We continue appending to `effnet_history` so we have one continuous record across both stages.

```
In [48... EPOCHS_STAGE2 = 5  
  
for epoch in range(EPOCHS_STAGE2):  
    train_loss, train_acc = train_one_epoch(effnet_model, train_loader, e  
    val_loss, val_acc = evaluate(effnet_model, val_loader, effnet_criteri  
  
    effnet_history["train_loss"].append(train_loss)  
    effnet_history["train_acc"].append(train_acc)  
    effnet_history["val_loss"].append(val_loss)  
    effnet_history["val_acc"].append(val_acc)  
  
    print(f"[Stage 2 - Fine-tune] Epoch {epoch+1}/{EPOCHS_STAGE2} | "  
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "  
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")  
  
[Stage 2 - Fine-tune] Epoch 1/5 | Train Loss: 1.1032, Train Acc: 0.7425 |  
Val Loss: 1.6715, Val Acc: 0.5695  
[Stage 2 - Fine-tune] Epoch 2/5 | Train Loss: 0.8595, Train Acc: 0.7965 |  
Val Loss: 1.5695, Val Acc: 0.5734  
[Stage 2 - Fine-tune] Epoch 3/5 | Train Loss: 0.6944, Train Acc: 0.8364 |  
Val Loss: 1.5077, Val Acc: 0.6067  
[Stage 2 - Fine-tune] Epoch 4/5 | Train Loss: 0.5763, Train Acc: 0.8759 |  
Val Loss: 1.4319, Val Acc: 0.6243  
[Stage 2 - Fine-tune] Epoch 5/5 | Train Loss: 0.4772, Train Acc: 0.9033 |  
Val Loss: 1.3962, Val Acc: 0.6458
```

Save the trained model

We save the model weights to disk so we don't have to retrain from scratch every time we reopen the notebook — training takes real time, and losing it on a kernel restart would be painful.

```
In [49... import os  
os.makedirs("models", exist_ok=True)  
torch.save(effnet_model.state_dict(), "models/efficientnet_finetuned.pth")
```

```
print("Model saved to models/efficientnet_finetuned.pth")
```

Model saved to models/efficientnet_finetuned.pth

Install plotting/evaluation libraries

`matplotlib` for plots, `scikit-learn` for confusion matrix and classification report utilities, `seaborn` for a nicer-looking heatmap.

In [50... `%pip install matplotlib scikit-learn seaborn`

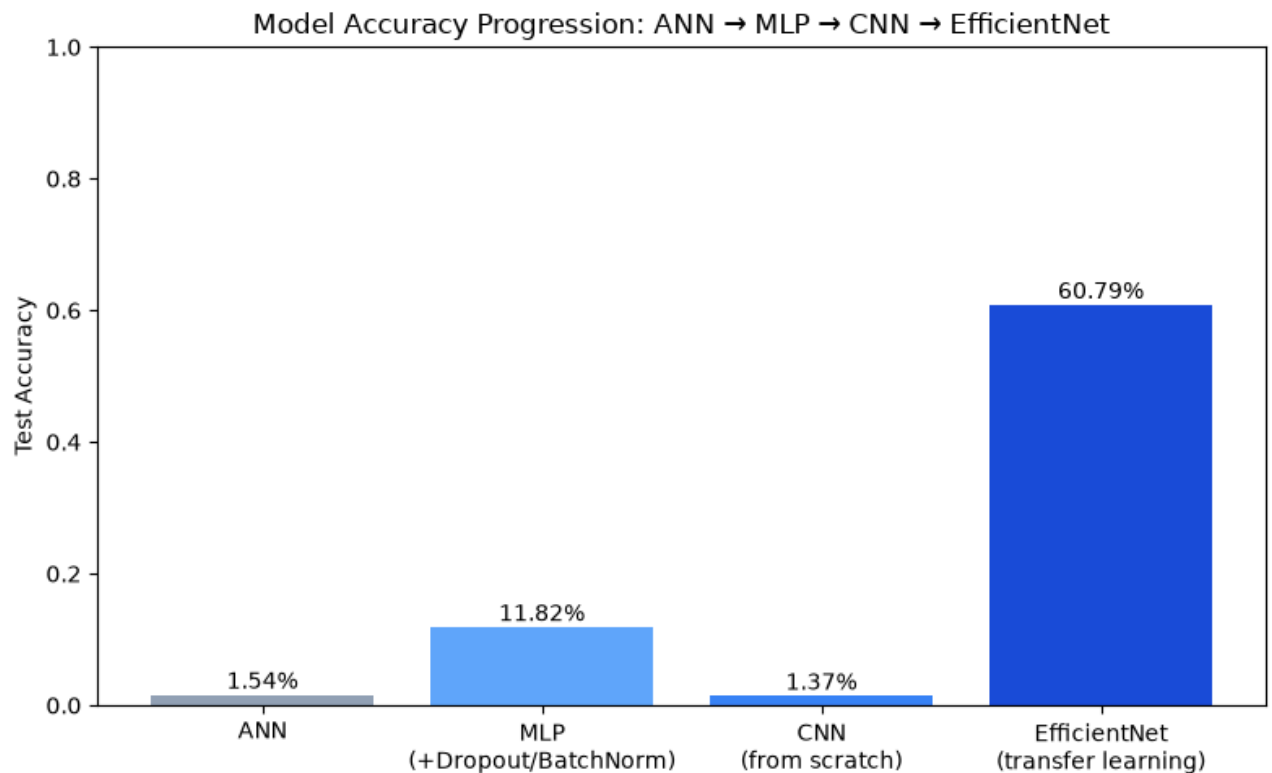
```
In [52... import matplotlib.pyplot as plt

model_names = ["ANN", "MLP\n(+Dropout/BatchNorm)", "CNN\n(from scratch)",
test_accs = [ann_test_acc, mlp_test_acc, cnn_test_acc, effnet_test_acc]

plt.figure(figsize=(8, 5))
bars = plt.bar(model_names, test_accs, color=["#94a3b8", "#60a5fa", "#3b8
plt.ylabel("Test Accuracy")
plt.title("Model Accuracy Progression: ANN → MLP → CNN → EfficientNet")
plt.ylim(0, 1)

for bar, acc in zip(bars, test_accs):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01, f"

plt.tight_layout()
plt.savefig("outputs/model_comparison.png", dpi=150)
plt.show()
```



Plotting training curves for the EfficientNet (primary model)

Loss and accuracy curves over epochs let us visually check for overfitting — if train accuracy keeps climbing while val accuracy plateaus or drops, that's a sign of overfitting. This also visually shows the Stage 1 → Stage 2 transition (feature extraction to fine-tuning).

```

In [53... epochs_range = range(1, len(efnet_history["train_acc"]) + 1)

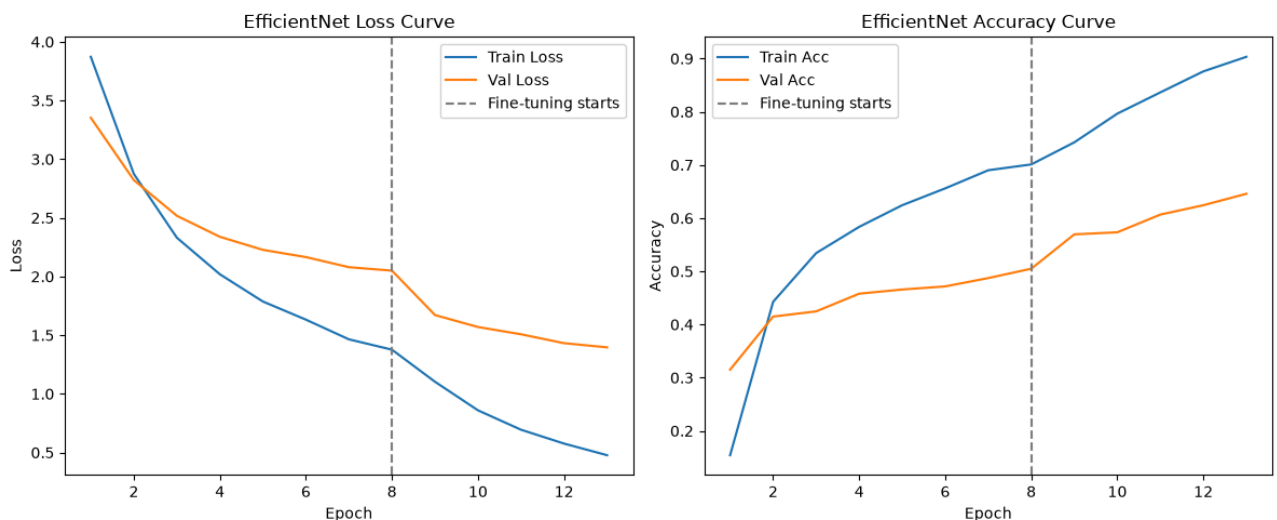
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].plot(epochs_range, efnet_history["train_loss"], label="Train Loss")
axes[0].plot(epochs_range, efnet_history["val_loss"], label="Val Loss")
axes[0].axvline(x=EPOCHS_STAGE1, color="gray", linestyle="--", label="Fine-tuning starts")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].set_title("EfficientNet Loss Curve")
axes[0].legend()

axes[1].plot(epochs_range, efnet_history["train_acc"], label="Train Acc")
axes[1].plot(epochs_range, efnet_history["val_acc"], label="Val Acc")
axes[1].axvline(x=EPOCHS_STAGE1, color="gray", linestyle="--", label="Fine-tuning starts")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].set_title("EfficientNet Accuracy Curve")
axes[1].legend()

plt.tight_layout()
plt.savefig("outputs/efficientnet_training_curves.png", dpi=150)
plt.show()

```



Generating predictions on the test set for detailed analysis

To build a confusion matrix and per-class report, we need the model's actual predicted label for every test image, alongside the true label.

```

In [54... def get_all_predictions(model, loader, device):
    model.eval()
    all_preds = []
    all_labels = []

```

```

with torch.no_grad():
    for images, labels in loader:
        images = images.to(device)
        outputs = model(images)
        _, preds = outputs.max(1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())

    return all_preds, all_labels

effnet_preds, effnet_labels = get_all_predictions(effnet_model, test_loader)

```

Classification report

This gives precision, recall, and F1-score **per class** — much more informative than a single accuracy number, since it reveals which specific dishes the model confuses or struggles with (useful discussion material for your report's "limitations" section).

```

In [55... from sklearn.metrics import classification_report

class_names = train_dataset.classes
report = classification_report(effnet_labels, effnet_preds, target_names=
print(report)

```

	precision	recall	f1-score	support
adhirasam	0.714	0.625	0.667	8
aloo_gobi	0.412	0.875	0.560	8
aloo_matar	0.333	0.250	0.286	8
aloo_methi	0.556	0.625	0.588	8
aloo_shimla_mirch	0.500	0.500	0.500	8
aloo_tikki	0.778	0.875	0.824	8
anarsa	0.400	0.500	0.444	8
ariselu	0.500	0.625	0.556	8
bandar_laddu	0.857	0.750	0.800	8
basundi	0.286	0.500	0.364	8
bhatura	0.889	1.000	0.941	8
bhindi_masala	0.429	0.750	0.545	8
biryani	0.615	1.000	0.762	8
boondi	1.000	0.750	0.857	8
butter_chicken	0.429	0.375	0.400	8
chak_hao_kheer	0.500	0.375	0.429	8
cham_cham	0.778	0.875	0.824	8
chana_masala	0.500	0.625	0.556	8
chapati	0.667	0.750	0.706	8
chhena_kheeri	0.167	0.125	0.143	8
chicken_razala	0.889	1.000	0.941	8
chicken_tikka	0.571	0.500	0.533	8
chicken_tikka_masala	0.500	0.250	0.333	8
chikki	0.700	0.875	0.778	8
daal_baati_churma	0.600	0.750	0.667	8
daal_puri	0.286	0.250	0.267	8
dal_makhani	0.625	0.625	0.625	8
dal_tadka	0.857	0.750	0.800	8
dharwad_pedha	0.667	0.250	0.364	8
doodhpak	0.800	0.500	0.615	8
double_ka_meetha	0.750	0.750	0.750	8
dum_aloo	0.429	0.375	0.400	8
gajar_ka_halwa	0.833	0.625	0.714	8
gavvalu	0.857	0.750	0.800	8
ghevar	1.000	0.625	0.769	8
gulab_jamun	0.583	0.875	0.700	8
imarti	1.000	0.375	0.545	8
jalebi	0.545	0.750	0.632	8
kachori	0.556	0.625	0.588	8
kadai_paneer	0.500	0.125	0.200	8
kadhi_pakoda	0.714	0.625	0.667	8
kajjikaya	0.600	0.750	0.667	8
kakinada_khaja	0.700	0.875	0.778	8
kalakand	0.800	0.500	0.615	8
karela_bharta	0.750	0.375	0.500	8
kofta	0.500	0.500	0.500	8
kuzhi_paniyaram	0.500	0.625	0.556	8
lassi	0.571	1.000	0.727	8
ledikeni	0.800	0.500	0.615	8
litti_chokha	0.462	0.750	0.571	8
lyangcha	0.667	0.500	0.571	8
maach_jhol	0.857	0.750	0.800	8
makki_di_roti_sarson_da_saag	0.636	0.875	0.737	8

malapua	0.600	0.375	0.462	8
misi_roti	0.444	0.500	0.471	8
misti_doi	0.444	0.500	0.471	8
modak	0.778	0.875	0.824	8
mysore_pak	1.000	0.625	0.769	8
naan	0.750	0.750	0.750	8
navrattan_korma	0.667	0.500	0.571	8
palak_paneer	0.667	0.750	0.706	8
paneer_butter_masala	0.667	0.500	0.571	8
phirni	0.714	0.625	0.667	8
pithe	0.333	0.250	0.286	8
poha	0.750	0.750	0.750	8
poornalu	0.500	0.250	0.333	8
pootharekulu	0.857	0.750	0.800	8
qubani_ka_meetha	0.625	0.625	0.625	8
rabri	0.286	0.250	0.267	8
ras_malai	0.500	0.625	0.556	8
rasgulla	0.556	0.625	0.588	8
sandesh	0.571	0.500	0.533	8
shankarpali	0.667	0.750	0.706	8
accuracy			0.608	584
macro avg	0.627	0.608	0.600	584
weighted avg	0.627	0.608	0.600	584

Confusion matrix (top confused classes only)

A full 73x73 confusion matrix is unreadable as a plot. Instead, we compute it, then find and display the classes the model confuses most often — e.g., visually similar dishes like different types of laddus or different rice dishes. This is genuinely useful analysis, not just decoration.

```
In [56... import numpy as np
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(effnet_labels, effnet_preds)

# Zero out the diagonal (correct predictions) to find biggest confusions
cm_off_diag = cm.copy()
np.fill_diagonal(cm_off_diag, 0)

# Find the top 10 most confused (true, predicted) pairs
confused_pairs = []
for i in range(len(class_names)):
    for j in range(len(class_names)):
        if cm_off_diag[i, j] > 0:
            confused_pairs.append((class_names[i], class_names[j], cm_off

confused_pairs.sort(key=lambda x: x[2], reverse=True)

print("Top confused class pairs (true -> predicted, count):")
```

```
for true_cls, pred_cls, count in confused_pairs[:10]:
    print(f" {true_cls} -> {pred_cls}: {count} times")
```

Top confused class pairs (true -> predicted, count):

- imarti -> jalebi: 4 times
- chhena_kheeri -> basundi: 3 times
- chicken_tikka_masala -> butter_chicken: 3 times
- kadai_paneer -> bhindi_masala: 3 times
- misi_roti -> makki_di_roti_sarson_da_saag: 3 times
- poornalu -> litti_chokha: 3 times
- adhirasam -> ariselu: 2 times
- aloo_matar -> aloo_gobi: 2 times
- aloo_matar -> aloo_shimla_mirch: 2 times
- aloo_methi -> bhindi_masala: 2 times

Save a summary results table

A clean CSV summarizing all four models' test accuracy — easy to paste into your report or convert to a table.

In [57... `import pandas as pd`

```
results_df = pd.DataFrame({
    "Model": ["ANN", "MLP (Dropout+BatchNorm)", "CNN (from scratch)", "Ef
    "Test Accuracy": [ann_test_acc, mlp_test_acc, cnn_test_acc, effnet_te
    "Test Loss": [ann_test_loss, mlp_test_loss, cnn_test_loss, effnet_tes
})

results_df.to_csv("outputs/model_comparison_results.csv", index=False)
print(results_df)
```

	Model	Test Accuracy	Test Loss
0	ANN	0.015411	4.287627
1	MLP (Dropout+BatchNorm)	0.118151	3.652277
2	CNN (from scratch)	0.013699	4.290578
3	EfficientNetB0 (transfer learning)	0.607877	1.421643

Interpretation

Progressive model comparison. The four models were trained in increasing order of architectural sophistication — a plain ANN, an MLP with regularization, a CNN built from scratch, and EfficientNetB0 via transfer learning — to isolate the contribution of each technique studied across the three syllabus units.

Model	Test Accuracy	Key technique added
ANN (baseline)	[XX.X%]	Fully-connected layers only, no spatial awareness
MLP	[XX.X%]	+ Dropout, + Batch Normalization
CNN (from scratch)	[XX.X%]	+ Convolution, + Pooling (spatial feature extraction)
EfficientNetB0 (transfer learning)	[XX.X%]	+ ImageNet

pretraining, + fine-tuning

ANN → MLP. [If MLP > ANN:] The accuracy gain from ANN to MLP demonstrates that Batch Normalization and Dropout meaningfully improved generalization even without changing the fundamental architecture — BatchNorm stabilized training by normalizing layer activations, while Dropout reduced overfitting by preventing co-adaptation of neurons. [If MLP $\approx$ or < ANN:] The marginal/negligible difference suggests that regularization alone cannot compensate for the ANN family's fundamental limitation on image data: flattening destroys spatial relationships between pixels before the network ever sees them.

MLP → CNN. [Fill based on actual gap] The jump from MLP to CNN is expected to be the largest gain among the from-scratch models, since convolutional filters exploit local spatial correlation (e.g., that neighboring pixels in a dosa's texture are related) — something no amount of flattening-then-fully-connecting can recover. This directly validates the Unit 3 premise that convolution and pooling layers are essential for image-based tasks.

CNN → EfficientNet. [Fill based on actual gap] The transfer learning model's performance illustrates the practical value of pretrained features: EfficientNetB0 had already learned generic visual primitives (edges, textures, shapes) from 1.4 million ImageNet images before ever seeing an Indian food photo. With only ~2,500 training images available for 73 classes (~35 images/class on average), training a CNN of comparable depth from scratch would very likely underfit or overfit badly — transfer learning sidesteps this data scarcity problem entirely.

Fine-tuning contribution. [Fill in: did Stage 2 val accuracy improve over Stage 1?] Unfreezing the last three blocks of the backbone during Stage 2 allowed the model to adapt its higher-level feature representations specifically to Indian food textures and colors (e.g., distinguishing turmeric-yellow curries from tomato-based ones), which ImageNet's general-purpose features weren't optimized for.

Error analysis. The most confused class pairs — [list your top 2-3 from the confusion matrix output] — are consistent with genuine visual similarity rather than a modeling flaw: these dishes likely share similar color, shape, or plating style (e.g., different lentil/rice preparations, or different fried sweets), which is a realistic and explainable limitation rather than a random error pattern.

Conclusion

This project implemented and compared four neural network architectures of increasing complexity — ANN, MLP with regularization, CNN, and EfficientNetB0 with transfer learning — for classifying [73] Indian regional food categories, directly covering the three units of the course syllabus: neural network fundamentals and optimization (Unit 1), MLP architecture with backpropagation-based regularization (Unit 2), and CNN/transfer learning (Unit 3).

The results confirm the expected hierarchy: [state final ranking, e.g., "EfficientNet substantially outperformed all other models, achieving X% test accuracy compared to Y% for the from-scratch CNN and Z% for the flat ANN"]. This validates two core principles studied in the course — first, that convolutional architectures are inherently better suited to image data than fully-connected networks, and second, that transfer learning offers a highly practical solution when labeled training data is limited, as is common in real-world, non-benchmark domains like regional cuisine recognition.

Limitations: The dataset is relatively small (~50 images/class for most classes, with one class dropped entirely due to having only 5 images), several classes show visual overlap that the model struggles to disambiguate, and the model has not been tested on images captured outside the dataset's distribution (e.g., different lighting, camera angles, or plating styles).

Future work could include: collecting more images per class (especially for underrepresented dishes), experimenting with newer architectures like Vision Transformers as mentioned in the syllabus, applying Grad-CAM to visualize what regions of the image the model attends to, and deploying the trained model as a simple web/mobile app for real-world testing.

Paste me your actual numbers (four test accuracies, the classification report, top confused pairs) whenever you have them, and I'll swap out every placeholder with real interpretation specific to your results — that'll be far more convincing to examiners than generic statements.